

3. Analizador Léxico

Descripción General

La única funcionalidad del lexer/scanner/tokenizer será pasar del código fuente a una secuencia de tokens, que utilizará la siguiente fase, el parser. Según la especificación de HLSL, el lenguaje sólo admite determinados rangos de caracteres en la tabla ASCII, por lo cuál si se encuentran bytes en el código fuente fuera de aquellos valores permitidos se generarán tokens de error. El lexer reconocerá todos los símbolos, operadores, palabras clave, y literales numéricos que admite el lenguaje, además de identificadores definidos por el usuario para nombres de variables, funciones, enumeraciones y structs. Además en la sección de palabras claves hay algunas que ya no se utilizan y están obsoletas, por lo cuál estas también generarían tokens de error, dando a entender al usuario que no las utilice.

Componentes Principales

- **Variables:** Un identificador es cualquier nombre dentro del código que no es una palabra clave, y cumple con la siguiente expresión regular: `[A-Za-z_]\w*`
- **Tipos de datos:**

Escalares	Enteros: bool, int, uint, dword Punto flotante: half, float, double
Vectores, Matrices	Vectores: float1, float2, float3, float4 Matrices: float2x2, float3x3, float4x4
Samplers, Texturas o Buffers	Samplers: Guardan el estado que describe cómo se accede a una textura. Texturas: Una matriz de píxeles con un formato específico. Desde el código sólo se puede acceder usando un sampler. Buffers: Son contenedores contiguos lineales que permiten acceso al azar de valores escalares, vectores y matrices de hasta 4 valores de 32 bits.
Estructuras, Definidos por el usuario	Estructuras: Agrupaciones de otras variables alineadas a ubicaciones de memoria que son potencias de 2.

	Definidos por el usuario: Aliases o renombramientos de otros tipos de datos.
Arreglos	Secuencia de otros tipos de datos, contigua en memoria.
Objetos de estado	Son estructuras de datos dentro del código de shader que corresponden a objetos en la CPU usados por la API de DirectX 12.

- **Operadores:**

Nombre del tipo de operador	Operadores
Operadores aditivos y multiplicativos	+, -, *, /, %
Operador de arreglo	[i]
Operadores de asignación	=, +=, -=, *=, /=, %=
Re-interpretaciones de memoria	asfloat(), asint()
Operaciones entre dígitos binarios	~, <<, >>, &, , ^, <<=, >>=, &=, =, ^=
Operadores booleanos	&&, , ?:
Operador de conversión	(tipo_de_dato)
Operador coma	,
Operadores de comparación	<, >, ==, !=, <=, >=
Operadores de prefijo y postfijo	++, --
Operador de estructura	.
Operadores unarios	!, -, +

- **Palabras Reservadas:** Existen más categorías y palabras claves de las que voy a mostrar, pero estas son las que voy a manejar (y además que sean válidas en el contexto de shaders de vértices y de píxeles).

Categoría	Palabras Clave
Tipos de datos primitivos	bool, int, uint, dword, half, float, double, void, string
Tipos de dato de vectores y matrices	vector, matrix, float2, float3, float4, float4x4
Literales booleanos y null	true, false, NULL
Declaraciones de tipos de datos	struct, typedef, namespace
Cualificadores de variables	const, static, uniform
Modificadores de parámetros de funciones	in, out, inout
Control de flujo	if, else, switch, case, default, for, while, do, break, continue, return, discard
Enlace de registros	register
Recursos de Buffer (Sólo se pueden leer, no modificar)	Buffer, StructuredBuffer, ByteAddressBuffer, Texture1D, Texture1DArray, Texture2D, Texture2DArray, Texture2DMS, Texture2DMSArray, Texture3D, TextureCube, TextureCubeArray
Recursos de Buffer (Se pueden leer y modificar)	RWBuffer, RWStructuredBuffer, RWByteAddressBuffer, RWTexture1D, RWTexture1DArray, RWTexture2D, RWTexture2DArray, RWTexture3D
Definición del comportamiento de acceso a una textura.	sampler, SamplerState, SamplerComparisonState
Buffers constantes	cbuffer, tbuffer
Modificadores de conversión normalizada de enteros a punto flotante	unorm, snorm

- **Comentarios:**

```
// Los comentarios de una línea se escriben así.  
/*  
 * Los comentarios de múltiples líneas  
 * se escriben de esta manera.  
*/
```

- **Delimitadores:** Todo los espacios en blanco son ignorados (\n, \t, ' ', etc.)

Delimitación de bloques de código y namespaces	{ }
Delimitación de clases, estructuras, y enumeraciones	{ };
Delimitación de argumentos de una función	()
Delimitación de plantillas (tipos de datos parametrizados)	< >
Delimitación de sentencias	;
Delimitación de atributos de funciones	[]